

P4 Non linear least squares

Bohaz

28.09.2026

1

We are interested in solving the non-linear least square problem. i.e. We wish to minimise the cost/error function

$$C(\mathbf{z}) = \frac{1}{2} \sum_i r_i(\mathbf{z})^2 = \frac{1}{2} |\mathbf{r}(\mathbf{z})|^2 \quad (1)$$

where $\mathbf{z}$ is a set of adjustable parameters, and r_i are errors between the ground truth and our model prediction, we could also call r_i the residue. Non linear just means r_i is not a linear function of z_k . Define the derivative of error with respect to parameter as Jacobian

$$J_{ik} = \frac{\partial r_i}{\partial z_k} \quad (2)$$

then one can show

$$\frac{\partial C}{\partial z_k} = \sum_i r_i J_{ik} = \sum_i J_{ki}^T r_i \quad (3)$$

in vector notation

$$\mathbf{g} = \nabla C = \mathbf{J}^T \mathbf{r} \quad (4)$$

our goal is to adjust all the z_k until we minimise C . The basic idea is that we start with some $\mathbf{z}$, then iteratively update $\mathbf{z} \leftarrow \mathbf{z} + \delta \mathbf{z}$ until we reach a minimum in C . The simplest way to do this is just move $\mathbf{z}$ in the direction of max negative slope of the cost function

$$\delta \mathbf{z} = -\alpha \mathbf{g} \quad (5)$$

where $\alpha > 0$ controls the step size, We are done if $\delta \mathbf{z}$ is small enough. This method is known as **gradient descent**, it is quite simple but only uses the local slope. In other words, how do we choose a correct α ? The slope does not tell us how far to move before the slope changes substantially. To illustrate what we mean by this, consider $C(z_1, z_2) = \frac{1}{2}(100z_1 + z_2)^2$, which has minimum $z_1 = z_2 = 0$ and slope is $\nabla C = (100z_1, z_2)$ and if we are currently at $(z_1 = 0.01, z_2 = 1)$, then $\nabla C = (1, 1)$, now we are in trouble if we use gradient descent, because setting $\alpha = 1$, will give $z_2 = 0$ (good), $z_1 = -0.99$ (bad, we massively overshoot) and if we set $\alpha = 0.01$ then $z_1 = 0$ (good), but $z_2 = 0.99$ (bad, it barely changed and has got a long way to go before reaching the minimum $z_2 = 0$), what is missing is the curvature information. We need α appropriate to each z_k based on the curvature information.

Gauss-Newton uses more information, for small $\delta \mathbf{z}$ it approximates

$$r_i(\mathbf{z} + \delta \mathbf{z}) \simeq r_i(\mathbf{z}) + \sum_k \frac{\partial r_i}{\partial z_k} \delta z_k \quad (6)$$

in vector notation

$$\mathbf{r}(\mathbf{z} + \delta\mathbf{z}) \simeq \mathbf{r}(\mathbf{z}) + \mathbf{J}\delta\mathbf{z} \quad (7)$$

then it chooses a step that minimises $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$, i.e. find the new step $\delta\mathbf{z}$ so the $\mathbf{z} + \delta\mathbf{z}$ gives lowest cost compared with others, differentiating $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$, with respect to z_k then using local linearity approximation Eq.(7) and setting to zero gives

$$\sum_i \left(r_i + \sum_k J_{ik} \delta z_k \right) J_{il} = 0 \quad (8)$$

in vector notation

$$\mathbf{J}^T(\mathbf{r} + \mathbf{J}\delta\mathbf{z}) = 0 \quad (9)$$

then rearrange yields

$$\mathbf{J}^T \mathbf{J} \delta\mathbf{z} = -\mathbf{J}^T \mathbf{r} = -\mathbf{g} \quad (10)$$

$\mathbf{J}^T \mathbf{J}$ is positive semi-definite because for some vector $\mathbf{v}$ we have $\mathbf{v}^T \mathbf{J}^T \mathbf{J} \mathbf{v} = |\mathbf{J}\mathbf{v}|^2 \geq 0$. To prevent various problems that arise $\mathbf{J}^T \mathbf{J}$ have eigenvalues as zeros, which can be numerically nasty (Such as solution not being unique and/or will result in dividing by zeros in the algorithm) we add a damping term, to guarantee $\mathbf{J}^T \mathbf{J}$ positive definite. This is known as **Levenberg-Marquardt**

$$(\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D}) \delta\mathbf{z} = -\mathbf{g} \quad (11)$$

where $D_{kk} = \max((\mathbf{J}^T \mathbf{J})_{kk}, \epsilon)$, where ϵ is a small positive number, controls the fractional damping of the diagonal entries and $\lambda > 0$ to ensure positive definiteness and non-singularity, we can refactorise it into a lower triangular matrix $\mathbf{L}$ and its transpose using Cholesky factorisation as explained in P2 and P3

$$\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D} = \mathbf{L} \mathbf{L}^T \quad (12)$$

then $\delta\mathbf{z}$ can be solved by doing a forward substitution followed by a back substitution. In the limit of weak damping $\lambda \rightarrow 0$, we recover Gauss-Newton method, if λ is large then

$$\delta z_k = -\lambda^{-1} (\mathbf{J}^T \mathbf{J})_{kk}^{-1} g_k \quad (13)$$

which is an adaptive step size version of gradient descent for each parameter where $\alpha_k^{-1} = \lambda (\mathbf{J}^T \mathbf{J})_{kk}$. With some initial trial λ' , if $C(\mathbf{z} + \delta\mathbf{z}'(\lambda')) \geq C$, then we should increase λ to take more cautious steps to find the right direction, before quickly wandering in the wrong direction. If $C(\mathbf{z} + \delta\mathbf{z}'(\lambda')) < C$, then we should lower λ , so that it takes larger steps as it is moving in the right direction to minimise C .

2

Three new problems arise due to the somewhat adhoc/arbitrary parameter λ , which we should feel just as uncomfortable as the original step size α , that we introduced to ensure the positive definiteness of the matrix that controls adaptive step size in gradient descent, so our Cholesky factorisation and solve can work as intended. The two problems are: 1. We just added the $\lambda \mathbf{D}$ term in Eq.(11) without deriving it, where does it come from, and 2. If λ is small, then $|\delta\mathbf{z}|$ is large, and will it cause the local linear approximation Eq.(7) to break down. So we look at these problems in this section. For simplicity, we will let $\mathbf{D} = \mathbf{1}$ to start off in this section.

Now consider minimising the the cost $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$ under the constraint that step size is small,

$$|\delta\mathbf{z}|^2 \leq \rho^2 \quad (14)$$

where $\rho > 0$ controls the permitted step size to ensure approximation in Eq.(7) is satisfied. This can be done using Lagrange's undetermined multiplier method with multiplier λ , however, it differs from usual Lagrange multiplier, because Eq.(14) is an inequality. The Lagrangian is

$$\mathcal{L}(\delta\mathbf{z}) = \frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2 + \frac{1}{2}\lambda(|\delta\mathbf{z}|^2 - \rho^2) \quad (15)$$

if $|\delta\mathbf{z}|^2 < \rho^2$, we do not need the constraint so $\lambda = 0$ in Eq.(15). But if $|\delta\mathbf{z}|^2 = \rho^2$, then this reduces back to a normal Lagrange undetermined multiplier method. But notice that in both cases, we have

$$\lambda(|\delta\mathbf{z}|^2 - \rho^2) = 0 \quad (16)$$

just like a normal Lagrange undetermined multiplier. Now consider the constrained minimum $|\mathbf{z}^*|^2 = \rho^2$, which satisfy

$$\left. \frac{\partial C}{\partial z_k} \right|_* + \lambda z_k^* = 0 \quad (17)$$

and consider some new proposed change $\mathbf{z}'$

$$z'_k = (1 - \epsilon)z_k^*, \quad 0 < \epsilon \ll 1 \quad (18)$$

thus

$$z'_k - z_k^* = -\epsilon z_k^* \quad (19)$$

Then because ϵ is as small as we like, the following linear expansion holds

$$\begin{aligned} C(\mathbf{z} + \delta\mathbf{z}') - C(\mathbf{z} + \delta\mathbf{z}^*) &= \sum_k \left. \frac{\partial C}{\partial z_k} \right|_* (\delta z'_k - \delta z_k^*) \\ &= -\epsilon \sum_k \left. \frac{\partial C}{\partial z_k} \right|_* z_k^* \\ &= \lambda \epsilon \sum_k (z_k^*)^2 \\ &= \lambda \epsilon \rho^2 \end{aligned} \quad (20)$$

if $\lambda < 0$, then it means the suggested new step can have lower cost, which contradicts with $\mathbf{z}^*$ is the constrained optimum, thus the converse must be true

$$\lambda \geq 0 \quad (21)$$

But the point of this is that we want there to be a constraint, so $\lambda = 0$ would be useless for us, then notice that minimising Eq.(15) with respect to δz_k is same as minimising

$$L(\delta\mathbf{z}) = \frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2 + \frac{1}{2}\lambda|\delta\mathbf{z}|^2 \quad (22)$$

but this is just a loss function with input $\mathbf{J}$, true output $\mathbf{r}$, weights $\delta\mathbf{z}$, and a L_2 /ridge regulariser with regularisation strength $\lambda > 0$. The point of λ punishes large step/weights

$\delta \mathbf{z}$ as it is more likely to reduce the validity of the local linear approximation Eq.(7). Then differentiation and setting to zero yields

$$(\mathbf{J}^T \mathbf{J} + \lambda \mathbf{1}) \delta \mathbf{z} = -\mathbf{g} \quad (23)$$

if we repeat the analysis for the constraint

$$\delta \mathbf{z}^T \mathbf{D} \delta \mathbf{z} \leq \rho^2 \quad (24)$$

which means we constrain the scaled step size $\delta z_k^{\text{scaled}} = \delta z_k / \sqrt{D_{kk}}$ is small. We then recover Eq.(11).

So in summary, the uncomfortableness of λ comes from the broadness of some implicit underlying Gaussian prior belief regarding the size of each step. Gauss-Newton method is iterative local linear least square, (maximisation of log likelihood) and Levenberg-Marquardt is local linear least square, with a regularisor to punish significant deviation from local linearity. (maximisation of log posterior) The loss function can also be interpreted, as the Lgrangian up to an additive constant, with the regularisor as the constraint and the regularisation strength as the undetermined multiplier. Lastly, the well known gradient descent is a crude uncontrolled non-adaptive approximation to the two methods above,

$$\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D} \simeq \alpha^{-1} \mathbf{1} \quad (25)$$

however, it is nevertheless perhaps the most simple and useful approximations.